

CSS Margin-Inline & Margin-Block

Complete Guide: Logical Properties for Internationalization

CSS Guide

May 10, 2026

Contents

1	Introduction	2
1.1	What are Margin-Inline and Margin-Block?	2
1.2	Why Logical Margin Properties Matter	2
1.2.1	Real-World Applications	2
1.2.2	Benefits	3
1.3	Browser Support	3
2	Core Concepts	4
2.1	Margin-Inline Property	4
2.1.1	Syntax	4
2.1.2	Value Meanings	4
2.2	Margin-Block Property	5
2.2.1	Syntax	5
2.2.2	Value Meanings	5
3	Practical Examples	6
3.1	Example 1: RTL-Safe Article Layout	6
3.2	Example 2: List Item Spacing	7
3.3	Example 3: Grid Layout Spacing	7
3.4	Example 4: Navigation with Margins	8
3.5	Example 5: Form Layout	9
4	Advanced Techniques	10
4.1	Combining with CSS Variables	10
4.2	Combining Logical Properties	11
4.3	Direction-Specific Overrides	11
5	Browser Support & Fallbacks	12
5.1	Progressive Enhancement	12
5.2	Feature Detection	12
6	Best Practices	13
6.1	Key Recommendations	13
6.1.1	1. Use for All New Projects	13
6.1.2	2. Combine with Direction Attribute	13
6.1.3	3. Maintain Consistency	13

7	Common Issues & Solutions	14
7.1	Issue 1: Centering Not Working	14
7.2	Issue 2: Spacing in RTL	14
8	Summary & Quick Reference	16
8.1	When to Use Logical Margins	16
8.2	Quick Reference	16
8.2.1	Property Mapping	16
8.2.2	Complete Example	17

Chapter 1

Introduction

1.1 What are Margin-Inline and Margin-Block?

The `margin-inline` and `margin-block` properties are CSS logical properties that control margins on the horizontal (inline) and vertical (block) axes respectively. Like their padding counterparts, these properties automatically adapt to the document's writing direction, enabling truly internationalized layouts that work in all languages without CSS modifications.

```
/* Traditional margin: physical properties */
margin-left: 30px;
margin-right: 30px;
margin-top: 20px;
margin-bottom: 20px;

/* Logical properties: direction-aware */
margin-inline: 30px;      /* left and right in LTR */
margin-block: 20px;      /* top and bottom */
```

Margin-inline and margin-block enable spacing that automatically reverses in RTL layouts and adapts to vertical writing modes without code duplication.

1.2 Why Logical Margin Properties Matter

1.2.1 Real-World Applications

- **Global Applications:** Auto-adapt to RTL and vertical languages
- **Spacing Systems:** Consistent spacing that respects direction
- **Component Libraries:** Write once, use everywhere
- **Arabic/Hebrew Sites:** Perfect RTL support without branches
- **CJK Languages:** Support vertical writing automatically
- **Accessibility:** Better support for diverse users

1.2.2 Benefits

- Automatic direction-aware spacing
- No duplicate CSS for RTL/LTR variants
- Cleaner, more maintainable code
- Single source of truth for spacing
- Future-proof responsive design
- Better internationalization support

1.3 Browser Support

Logical margin properties are well-supported:

Browser	Version	Status
Chrome/Edge	v69+	Full Support
Firefox	v68+	Full Support
Safari	v12+	Full Support
Opera	v56+	Full Support
IE 11	N/A	Not Supported

Chapter 2

Core Concepts

2.1 Margin-Inline Property

The `margin-inline` property sets margins on both sides of the inline axis, adapting to writing direction.

2.1.1 Syntax

```
/* Equal margin on inline start and end */
margin-inline: 30px;

/* Inline-start and inline-end */
margin-inline: 20px 40px;

/* Longhand form */
margin-inline-start: 20px;
margin-inline-end: 40px;

/* With units */
margin-inline: 1rem;
margin-inline: 1em 2em;

/* Negative margins */
margin-inline: -10px;
margin-inline: -5px 15px;

/* Auto for centering */
margin-inline: auto;
```

2.1.2 Value Meanings

Syntax	LTR Behavior	RTL Behavior
30px	margin-left: 30px, margin-right: 30px	Reversed
20px 40px	margin-left: 20px, margin-right: 40px	Reversed
auto	Centers horizontally	Centers horizontally

2.2 Margin-Block Property

The `margin-block` property sets margins on both sides of the block axis.

2.2.1 Syntax

```
/* Equal margin on block start and end */
margin-block: 20px;

/* Block-start and block-end */
margin-block: 10px 30px;

/* Longhand form */
margin-block-start: 10px;
margin-block-end: 30px;

/* With units */
margin-block: 1rem;
margin-block: 1.5em 2em;

/* Negative margins */
margin-block: -5px;
margin-block: 10px -10px;

/* Zero margin */
margin-block: 0;
```

2.2.2 Value Meanings

Syntax	Horizontal Text	Vertical Text
20px	margin-top: 20px, margin-bottom: 20px	Rotated
10px 30px	margin-top: 10px, margin-bottom: 30px	Rotated
0	No top/bottom margin	No block margins

Chapter 3

Practical Examples

3.1 Example 1: RTL-Safe Article Layout

```
/* Article container - works in all languages */
article {
    max-width: 800px;
    margin-inline: auto;    /* Centers in LTR and RTL */
    margin-block: 40px;
}

/* Article sections */
section {
    margin-block: 30px;
    margin-inline: 0;
}

/* Headings */
h1 {
    margin-block: 20px 10px;
    margin-inline: 0;
}

h2 {
    margin-block: 15px 8px;
    margin-inline: 0;
}

/* Paragraphs */
p {
    margin-block: 0 1em;
    margin-inline: 0;
}

/* Works perfectly in:
   English (LTR), Arabic (RTL), Hebrew (RTL),
   Chinese (Vertical), Japanese (Vertical)
*/
```


3.2 Example 2: List Item Spacing

```
/* List container */
.list {
  list-style: none;
  margin: 0;
  padding: 0;
}

/* List item */
.list-item {
  margin-block: 8px;
  margin-inline: 0;
  padding-inline: 24px;
  position: relative;
}

/* List marker/bullet */
.list-item::before {
  content: '•';
  position: absolute;
  margin-inline-start: 0;
  margin-inline-end: 12px;
}

/* Nested lists */
.list-item .list {
  margin-inline-start: 20px;
  margin-block: 8px;
}

/* Auto-adjusts for RTL without code changes */
```

3.3 Example 3: Grid Layout Spacing

```
/* Grid container */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 20px;
}

/* Grid items */
.grid-item {
  margin-block: 0;
  margin-inline: 0;
}

/* Card with margin */
.card {
  margin-block: 16px;
}
```

```
    margin-inline: 12px;
    padding: 20px;
    background: white;
    border-radius: 8px;
}

/* Featured item with extra margin */
.card.featured {
    margin-block: 24px;
    margin-inline: 8px;
}

/* Margins adapt direction automatically */
```

3.4 Example 4: Navigation with Margins

```
/* Navigation container */
nav {
    margin-block: 0;
    margin-inline: 0;
    background: #667eea;
}

/* Navigation list */
.nav-list {
    display: flex;
    list-style: none;
    margin: 0;
    padding: 0;
}

/* Navigation items */
.nav-item {
    margin-block: 0;
    margin-inline: 16px;
}

/* First item: no margin start */
.nav-item:first-child {
    margin-inline-start: 0;
}

/* Last item: no margin end */
.nav-item:last-child {
    margin-inline-end: 0;
}

/* Sub-menu */
.submenu {
    margin-block: 8px;
    margin-inline-start: 12px;
}
```

```
/* Works in LTR and RTL without changes */
```

3.5 Example 5: Form Layout

```
/* Form group */
.form-group {
    margin-block: 20px;
    margin-inline: 0;
}

/* Label */
label {
    display: block;
    margin-block: 0 8px;
    margin-inline: 0;
    font-weight: bold;
}

/* Input */
input[type="text"],
input[type="email"],
textarea {
    width: 100%;
    margin-block: 0;
    margin-inline: 0;
    padding: 8px 12px;
    border: 1px solid #ddd;
    border-radius: 4px;
}

/* Button */
button {
    margin-block: 20px 0;
    margin-inline: 0;
    padding: 10px 20px;
    background: #667eea;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}

/* Error message */
.error {
    margin-block: 8px;
    margin-inline: 0;
    color: #e74c3c;
    font-size: 12px;
}

/* RTL forms work perfectly */
```

Chapter 4

Advanced Techniques

4.1 Combining with CSS Variables

```
/* Define margin scales */
:root {
  --margin-0: 0;
  --margin-xs: 4px;
  --margin-sm: 8px;
  --margin-md: 16px;
  --margin-lg: 24px;
  --margin-xl: 32px;
  --margin-2xl: 48px;
}

/* Use in components */
.section {
  margin-block: var(--margin-xl);
  margin-inline: 0;
}

.card {
  margin-block: var(--margin-md);
  margin-inline: var(--margin-sm);
}

.button {
  margin-block: var(--margin-sm);
  margin-inline: var(--margin-md);
}

/* Responsive margins */
@media (max-width: 768px) {
  :root {
    --margin-lg: 16px;
    --margin-xl: 24px;
  }
}
```

4.2 Combining Logical Properties

```
/* Use both margin and padding logically */
.container {
  margin-block: 40px;
  margin-inline: auto;
  padding-block: 20px;
  padding-inline: 30px;
}

/* No need to specify left/right separately */
.box {
  margin-inline: 20px;      /* Both sides */
  margin-block: 10px;      /* Both sides */
}

/* Asymmetric spacing */
.item {
  margin-inline-start: 16px;
  margin-inline-end: 24px;
  margin-block-start: 10px;
  margin-block-end: 20px;
}
```

4.3 Direction-Specific Overrides

```
/* Base: direction-agnostic */
.sidebar {
  margin-inline: 0;
  margin-block: 0;
}

/* RTL-specific adjustment */
[dir="rtl"] .sidebar {
  margin-inline-end: 20px;
}

/* LTR-specific adjustment */
[dir="ltr"] .sidebar {
  margin-inline-start: 20px;
}

/* Or use auto for centering */
.centered-box {
  margin-inline: auto;
  margin-block: 40px;
}
```

Chapter 5

Browser Support & Fallbacks

5.1 Progressive Enhancement

```
/* Fallback for older browsers */
.element {
    margin: 20px 30px;
}

/* Enhanced for modern browsers */
@supports (margin-inline: 0) {
    .element {
        margin-block: 20px;
        margin-inline: 30px;
    }
}
```

5.2 Feature Detection

```
/* Check support */
@supports (margin-inline: 0) {
    /* Use logical properties */
    .component {
        margin-block: 20px;
        margin-inline: 30px;
    }
}

@supports not (margin-inline: 0) {
    /* Use physical properties */
    .component {
        margin: 20px 30px;
    }
}
```

Chapter 6

Best Practices

6.1 Key Recommendations

6.1.1 1. Use for All New Projects

```
/* Good: Future-proof */
.component {
  margin-block: 20px;
  margin-inline: 30px;
}

/* Acceptable: Works but not i18n-ready */
.component {
  margin: 20px 30px;
}
```

6.1.2 2. Combine with Direction Attribute

```
/* Always set HTML dir attribute */
/* <html dir="ltr"> */
/* <html dir="rtl"> */

/* CSS automatically adapts */
.element {
  margin-block: 10px;
  margin-inline: 20px;
}
```

6.1.3 3. Maintain Consistency

```
/* Consistent margin system */
.spacing-0 { margin-block: 0; margin-inline: 0; }
.spacing-1 { margin-block: 4px; margin-inline: 4px; }
.spacing-2 { margin-block: 8px; margin-inline: 8px; }
.spacing-3 { margin-block: 16px; margin-inline: 16px; }
.spacing-4 { margin-block: 24px; margin-inline: 24px; }
```

Chapter 7

Common Issues & Solutions

7.1 Issue 1: Centering Not Working

Problem: Element not centered with `margin-inline: auto`.

Cause: Element width not set or container too small.

Solution:

```
/* Good: Width defined and centered */
.container {
  width: 80%;
  max-width: 1200px;
  margin-inline: auto;
  margin-block: 40px;
}

/* With padding */
.container {
  width: 90vw;
  max-width: 1200px;
  margin-inline: auto;
  padding-inline: 20px;
}
```

7.2 Issue 2: Spacing in RTL

Problem: Margins wrong in RTL layout.

Cause: Using physical properties or incorrect values.

Solution:

```
/* Wrong */
.item { margin-left: 20px; }

/* Correct */
.item { margin-inline-start: 20px; }

/* Or symmetric */
.item { margin-inline: 20px; }
```



```
/* Automatically correct in RTL */
```

Chapter 8

Summary & Quick Reference

8.1 When to Use Logical Margins

Use logical margins when:

- Building multilingual apps
- Supporting RTL languages
- Creating reusable components
- Building design systems
- Future-proofing CSS

8.2 Quick Reference

8.2.1 Property Mapping

```
/* Physical to Logical Conversion */

/* Horizontal margins */
margin-left: 20px; →    margin-inline-start: 20px;
margin-right: 20px; →   margin-inline-end: 20px;
margin-left + right →   margin-inline: 20px;

/* Vertical margins */
margin-top: 10px; →     margin-block-start: 10px;
margin-bottom: 10px; →  margin-block-end: 10px;
margin-top + bottom →   margin-block: 10px;

/* All margins */
margin: 10px 20px; →    margin-block: 10px; margin-inline: 20px;

/* Centering */
margin: 0 auto; →       margin-inline: auto;
```

8.2.2 Complete Example

```
/* Modern component with logical properties */
.section {
  max-width: 1200px;
  margin-inline: auto;
  margin-block: 40px;
  padding-inline: 20px;
}

.card {
  margin-block: 16px;
  margin-inline: 0;
}

.button {
  margin-block: 12px;
  margin-inline-start: 0;
  margin-inline-end: auto;
}

/* Works perfectly in:
   English (LTR), Arabic (RTL), Hebrew (RTL)
   Chinese (Vertical), Japanese (Vertical)
*/
```

Thank you for reading!

For more information, visit [MDN Web Docs](#) and [web.dev](#)